

Python E-Course

Module 14 — Frequency Domain (DFT)

Detailed Notes

Module Overview

Level: Advanced

Duration: ~1 week

Prerequisites: Modules 1-13

What You'll Learn

- Why frequencies matter (1D audio analogy → 2D images).
- Compute and visualise the 2D DFT with `cv2.dft` / `np.fft`.
- Design low-pass, high-pass, and band-pass filters in the frequency domain.
- Build notch filters to remove periodic noise.
- Reason about spatial filtering as frequency multiplication.

Lessons

#	Lesson	Duration
1	1D Fourier — Audio Analogy	30 min
2	2D DFT — Magnitude Spectrum	35 min
3	Low-Pass & High-Pass Filters	30 min
4	Notch Filters — Periodic Noise	30 min
5	Spatial ↔ Frequency Equivalence	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 15: Image Restoration & Noise

Lesson 1: 1D Fourier — Audio Analogy

Module: 14 — Frequency Domain (DFT)

Duration: 30 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Explain frequency intuitively via the audio analogy.
- Compute a 1D DFT with NumPy.
- Read a magnitude spectrum.
- Filter a 1D signal by zeroing frequencies.

Prerequisites

- Modules 1-13, basic high-school math (sine waves).

1. Why This Matters

Frequencies are easier to grasp in 1D (audio) than in 2D (images). Once you've seen "a low rumble + a high whistle = a recorded sound", the 2D version ("slow gradient + fine texture = a photo") falls into place.

2. Concept

2.1 A signal is a sum of sines

Any signal can be decomposed into a sum of sine waves of different frequencies. That's what Fourier showed.

In sound:

- Low frequencies = bass (slow oscillations).
- High frequencies = treble (fast oscillations).
- A musical note = many frequencies summed together.

In a 1D image row:

- Low frequencies = slow brightness changes (the overall gradient, smooth shading).
- High frequencies = fast brightness changes (edges, fine texture, noise).

2.2 The Discrete Fourier Transform (DFT)

For a length-N signal $x[n]$:

$$X[k] = \sum_n x[n] \cdot \exp(-2\pi i \cdot kn/N) \quad \text{for } k = 0..N-1$$

$X[k]$ is complex: magnitude (how strong frequency k is) and phase (where its peaks are).

You'll almost never compute this by hand. NumPy's `np.fft.fft` does it.

2.3 The magnitude spectrum

```
X = np.fft.fft(x)
mag = np.abs(X)
```

Peaks in mag tell you which frequencies dominate. A pure sine wave produces one tall peak; a noisy signal has many small peaks.

2.4 Frequency bins

$X[0]$ is the DC component (average / zero frequency). $X[1]$ is the lowest non-zero frequency. $X[N/2]$ is the highest representable frequency (Nyquist). $X[k]$ for $k > N/2$ is the negative-frequency mirror.

For an image row of 1024 pixels, that's 512 distinct frequencies.

2.5 Filtering in frequency space

To remove fast oscillations from a signal:

- Compute the DFT.
- Zero out the high-frequency coefficients.
- Inverse-DFT back to time/space.

The result is a low-pass-filtered signal — equivalent to a smoothing filter applied in time/space.

2.6 fftshift

By default fft puts the DC at index 0. For plotting and centered filtering, shift it to the middle:

```
X_shifted = np.fft.fftshift(X)      # DC now at N/2
```

3. Code Examples

Example 1: Pure sine + DFT

```
import numpy as np
import matplotlib.pyplot as plt

N = 256
t = np.arange(N)
x = 1.5 * np.sin(2 * np.pi * 8 * t / N) + 0.8 * np.sin(2 * np.pi * 20 * t / N)

X = np.fft.fft(x)
mag = np.abs(X)

fig, ax = plt.subplots(2, 1, figsize=(8, 5))
ax[0].plot(x); ax[0].set_title("signal")
ax[1].plot(mag[:N/2]); ax[1].set_title("magnitude (positive freq half)")
ax[1].set_xlabel("frequency bin")
plt.tight_layout(); plt.show()
print("peaks at:", np.argsort(mag[:N/2])[-2:][::-1])
```

Expected Output: Top: a wavy signal. Bottom: two tall spikes at bins 8 and 20 (the frequencies we put in). Console prints peaks at: [8, 20].

Example 2: Noisy signal → low-pass via DFT

```
import numpy as np
import matplotlib.pyplot as plt

N = 256
t = np.arange(N)
clean = np.sin(2 * np.pi * 5 * t / N)
np.random.seed(0)
noisy = clean + 0.4 * np.random.randn(N)

X = np.fft.fft(noisy)
# Keep only frequencies 0..15 (and their mirrors at N-15..N-1)
mask = np.zeros(N)
mask[:16] = 1; mask[-15:] = 1
X_lp = X * mask
clean_est = np.real(np.fft.ifft(X_lp))

fig, ax = plt.subplots(2, 1, figsize=(8, 5))
ax[0].plot(noisy, label="noisy"); ax[0].plot(clean, label="true clean", alpha=0.5);
ax[0].legend()
ax[1].plot(clean_est); ax[1].set_title("recovered via low-pass DFT")
plt.tight_layout(); plt.show()
```

Expected Output: Top: jittery noisy signal overlaid with the smooth target. Bottom: recovered signal nearly matches the clean target.

Example 3: Inspect an image row's spectrum

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera()
row = img[256].astype(np.float32) # row through middle

X = np.fft.fft(row)
mag = np.abs(np.fft.fftshift(X)) # shifted: 0 freq in the middle

fig, ax = plt.subplots(2, 1, figsize=(8, 5))
ax[0].plot(row); ax[0].set_title("middle row intensity")
ax[1].plot(mag); ax[1].set_title("|DFT|, shifted")
plt.tight_layout(); plt.show()
print("DC magnitude:", mag[len(mag)//2])
```

Expected Output: Top: gray-level profile along the row. Bottom: a symmetric spectrum, peaked at the centre (DC = average intensity $\times N$), with smaller side peaks for textures.

Example 4: Verify FFT round-trip

```
import numpy as np
```

```

x = np.random.randn(64)
back = np.fft.ifft(np.fft.fft(x))
print("max imag part:", np.abs(back.imag).max())
print("max real diff:", np.abs(x - back.real).max())

```

Expected Output: Both numbers tiny ($\sim 1e-15$) — `ifft(fft(x))` recovers `x` exactly (up to floating-point error).

4. Parameter Sensitivity

(Conceptual lesson — see L3 for filter design parameters.)

5. Common Mistakes

Mistake	What Happens	Fix
Plot real part of <code>np.fft.fft</code> directly	Negative values, confusing	Use <code>np.abs</code> for magnitude
Forget Hermitian symmetry	Confused by "negative frequencies"	They're the mirror; for real signals, equivalent info
Filter without symmetry	Result has imaginary part	Symmetric masks keep result real

6. Practice Tasks

- Build a signal with sines at frequencies 5 and 25; verify FFT shows peaks there.
- Add Gaussian noise to a sine wave; recover it by zeroing high frequencies.
- Inspect the DFT of one row of `camera()`; print the DC magnitude.

7. Key Takeaways

- Any signal = sum of sines at different frequencies.
- DFT: signal $\leftrightarrow$ complex spectrum (`np.fft.fft` / `np.fft.ifft`).
- `np.abs` for magnitude; `np.fft.fftshift` to centre DC.
- Filtering in frequency = zero out unwanted frequencies, IFFT back.

8. What's Next

Same idea in 2D — the DFT of an image.

Lesson 2: 2D DFT — Magnitude Spectrum

Module: 14 — Frequency Domain (DFT)

Duration: 35 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Compute the 2D DFT of an image with `np.fft.fft2` and `cv2.dft`.
- Visualise the magnitude spectrum (log-scaled, shifted).
- Read the centred spectrum: where are low/high frequencies?
- Interpret common visual patterns in the spectrum.

Prerequisites

- Module 14 Lesson 1

1. Why This Matters

Once you can read a 2D spectrum, you can diagnose images at a glance: blurry → bright centre only, fine texture → energy at the edges, periodic noise → bright spots off-centre. Frequency-domain filtering is then "draw a mask in the spectrum and invert."

2. Concept

2.1 2D DFT — two passes of 1D DFT

The 2D DFT decomposes a (H, W) image into a 2D grid of complex coefficients of the same shape. Each coefficient (u, v) says "how much sine-wave content at horizontal frequency u and vertical frequency v is in the image".

Mathematically:

$$F(u, v) = \sum_x \sum_y I(x, y) \cdot \exp(-2\pi i \cdot (ux/W + vy/H))$$

But you'll never compute this by hand. Use:

```
F = np.fft.fft2(img)          # complex (H, W)
```

2.2 Centring with `fftshift`

By default, frequency (0, 0) (the DC) lives at array position [0, 0]. For viewing and centred masks, shift it to the middle:

```
Fs = np.fft.fftshift(F)      # DC at (H//2, W//2)
```

Now low frequencies sit near the centre and high frequencies are at the corners (of the shifted array, that means farther from centre).

2.3 Magnitude spectrum

```
mag = np.abs(Fs)
```

Values span many orders of magnitude — the DC alone is often $>10^6$ while textures are 10^2 . Use log for display:

```
mag_vis = np.log1p(mag)
mag_vis = cv2.normalize(mag_vis, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

2.4 Reading the spectrum

Pattern in spectrum	What it means about the image
Bright DC, dim elsewhere	Mostly uniform / very blurry
Bright cross + DC	Strong horizontal AND vertical edges (a building scene)
Bright diagonal smear	Diagonal edges / a tilted dominant texture
Bright dots off-centre	Periodic patterns (screens, fences, stripes)
Energy spread to corners	Lots of fine texture / noise

The spectrum is symmetric about the centre for real-valued images (Hermitian symmetry).

2.5 Inverse DFT

```
F = np.fft.fft2(img)
img_back = np.real(np.fft.ifft2(F))
```

Within rounding, identical to the input. Filtering happens between `fft2` and `ifft2`.

2.6 OpenCV vs NumPy

API	Notes
<code>np.fft.fft2 / ifft2</code>	Simplest; returns complex array
<code>cv2.dft(..., flags=cv2.DFT_COMPLEX_OUTPUT)</code>	Faster on large images, returns (H, W, 2) real+imag
<code>cv2.idft</code>	Inverse

For learning, stick with NumPy. For production with big images, OpenCV.

3. Code Examples

Example 1: Spectrum of camera()

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
Fs = np.fft.fftshift(F)
mag_vis = np.log1p(np.abs(Fs))
mag_vis = cv2.normalize(mag_vis, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(img, cmap="gray"); ax[0].set_title("image"); ax[0].axis("off")
ax[1].imshow(mag_vis, cmap="gray"); ax[1].set_title("log |DFT|, shifted");
ax[1].axis("off")
plt.tight_layout(); plt.show()
print("F shape:", F.shape, "dtype:", F.dtype)
```


Expected Output (visual): Left — cameraman. Right — bright spot in the middle (DC), with horizontal and vertical bright lines crossing it (from building edges) and a dim "haze" away from centre (other textures). Console prints (512, 512) complex128.

Example 2: Round-trip

```
import numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
back = np.real(np.fft.ifft2(F))
print("max abs diff:", np.abs(back - img).max())
```

Expected Output: max abs diff: <1e-9 — perfect round-trip up to floating point.

Example 3: Compare spectra of different content

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

def spectrum(img):
    F = np.fft.fft2(img.astype(np.float32))
    Fs = np.fft.fftshift(F)
    v = np.log1p(np.abs(Fs))
    return cv2.normalize(v, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 4)
noisy = np.clip(img + np.random.normal(0, 25, img.shape), 0, 255).astype(np.uint8)

fig, ax = plt.subplots(2, 3, figsize=(12, 7))
for col, (name, x) in enumerate([("sharp", img), ("blurred", blur), ("noisy",
noisy)]):
    ax[0, col].imshow(x, cmap="gray"); ax[0, col].set_title(name); ax[0,
col].axis("off")
    ax[1, col].imshow(spectrum(x), cmap="gray"); ax[1, col].axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Blurred image has spectrum concentrated near the centre (highs gone). Noisy image has noticeably brighter outer regions (highs amplified). Sharp image is in between.

Example 4: OpenCV DFT

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
dft = cv2.dft(img, flags=cv2.DFT_COMPLEX_OUTPUT) # shape (H, W, 2)
dft_shift = np.fft.fftshift(dft, axes=(0, 1))
mag = cv2.magnitude(dft_shift[..., 0], dft_shift[..., 1])
mag_vis = cv2.normalize(np.log1p(mag), None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
```

```
cv2.imwrite("cam_spectrum_cv.png", mag_vis)
print("dft shape:", dft.shape)
```

Expected Output: dft shape: (512, 512, 2) and a saved spectrum visually similar to Example 1's right panel.

4. Parameter Sensitivity

Choice	Effect
Pre-resize to power of 2	DFT slightly faster (negligible for fft2)
np.log vs np.log1p	log1p handles zeros gracefully
fftshift vs not	Just visualisation convenience

5. Common Mistakes

Mistake	What Happens	Fix
Display np.abs(F) directly	Huge DC overshadows everything	Take log1p then normalize
Forget fftshift	DC stuck in the corner; harder to interpret	Always shift for display
Treat np.fft.fft2(img) as float	It's complex	Use np.abs for magnitude
ifft2 returns complex	.real to drop tiny imaginary residue	np.real(ifft2(...))

6. Practice Tasks

- Compute and visualise the log-magnitude spectrum of camera().
- Do the same for a Gaussian-blurred copy ($\sigma=4$); compare brightness distribution.
- Compute spectrum via OpenCV cv2.dft; verify magnitude matches NumPy.

7. Key Takeaways

- 2D DFT: `np.fft.fft2(img)` $\rightarrow$ complex (H, W).
- Display: `log1p(|fftshift(F)|)` then `cv2.normalize`.
- Centre = low frequencies; corners = high frequencies.
- Spectrum patterns diagnose what's in the image.

8. What's Next

Filtering — drawing low-pass and high-pass masks in the spectrum and inverting back to the image.

Lesson 3: Low-Pass & High-Pass Filters

Module: 14 — Frequency Domain (DFT)

Duration: 30 minutes

Difficulty: Advanced

Learning Objectives

- Design ideal low-pass and high-pass masks in frequency space.
- Apply Gaussian and Butterworth alternatives that avoid ringing.
- Inverse-DFT back to the image.
- Recognise ringing and how to control it.

Prerequisites

- Module 14 Lesson 2

1. Why This Matters

Frequency-domain filtering gives you fine control over what spatial scales survive. It's slower than a convolution for everyday blurs but irreplaceable for unusual filters (band-stops, custom shapes) and for understanding why spatial filters do what they do.

2. Concept

2.1 The pipeline

- $F = \text{fft2}(\text{image})$, then $F_s = \text{fftshift}(F)$.
- Multiply F_s by a mask $H(u, v)$ — same shape as F_s .
- Unshift: $F_2 = \text{ifftshift}(F_s * H)$.
- Inverse DFT: $\text{out} = \text{real}(\text{ifft2}(F_2))$.
- Clip to display range.

Ideal LP keeps a centred disk of radius D_0 ; ideal HP keeps the complement (centre zeroed out).

2.2 Distance-from-centre map

A reusable helper:

```
def distance_map(shape):
    h, w = shape
    cy, cx = h // 2, w // 2
    y, x = np.ogrid[:h, :w]
    return np.sqrt((y - cy)**2 + (x - cx)**2)
```

2.3 Ideal filters (sharp boundary)

```
D = distance_map(img.shape)
mask_lp = (D <= D0).astype(np.float32)      # ideal LP
mask_hp = 1.0 - mask_lp                     # ideal HP
```

Problem: ideal filters cause ringing — visible ripples around edges. The hard boundary in frequency space produces a sinc pattern in image space.

2.4 Gaussian filters (smooth boundary, no ringing)

```
mask_lp = np.exp(-(D**2) / (2 * sigma**2))
mask_hp = 1.0 - mask_lp
```

Smooth roll-off → no ringing. Use these by default.

2.5 Butterworth filters (controlled steepness)

```
mask_lp = 1.0 / (1.0 + (D / D0)**(2 * n))    # order n
```

Higher n → steeper transition (closer to ideal, more ringing). n=2 is a good default.

2.6 Reading the result

LP: removes fine detail → smooth, blurry image. Roughly equivalent to spatial Gaussian blur with related σ .

HP: removes the slow-changing content → only edges and texture visible, centred around zero (offset for display).

2.7 D0 — the cutoff

D0 is the radius (in spectrum pixels) at which the filter transitions. For a 512×512 image:

- D0 = 30 → strong LP (removes most detail)
- D0 = 80 → moderate LP
- D0 = 200 → mild LP

For HP, same numbers — but they mean "remove frequencies below this radius".

3. Code Examples

Example 1: Ideal vs Gaussian LP — see ringing

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
Fs = np.fft.fftshift(F)

h, w = img.shape
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)

D0 = 40
mask_ideal = (D <= D0).astype(np.float32)
mask_gaussian = np.exp(-(D**2) / (2 * D0**2))
```

```

def filt(Fs, mask):
    F2 = np.fft.ifftshift(Fs * mask)
    return np.clip(np.real(np.fft.ifft2(F2)), 0, 255).astype(np.uint8)

lp_ideal = filt(Fs, mask_ideal)
lp_gauss = filt(Fs, mask_gaussian)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img.astype(np.uint8), cmap="gray"); ax[0].set_title("original")
ax[1].imshow(lp_ideal, cmap="gray"); ax[1].set_title("ideal LP
(ringing)")
ax[2].imshow(lp_gauss, cmap="gray"); ax[2].set_title("Gaussian LP")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()

```

Expected Output (visual): Ideal LP has visible halos / ripples around the cameraman's silhouette. Gaussian LP is just smoothly blurred — no ringing.

Example 2: HP filter — only edges remain

```

import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
Fs = np.fft.fftshift(F)

h, w = img.shape
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)
D0 = 30
mask_hp = 1.0 - np.exp(-(D**2) / (2 * D0**2))

F2 = np.fft.ifftshift(Fs * mask_hp)
hp = np.real(np.fft.ifft2(F2))
hp_vis = cv2.normalize(hp, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("cam_hp.png", hp_vis)
print("HP range:", hp.min(), hp.max())

```

Expected Output (visual): Edges of the cameraman + buildings pop out as bright/dark traces on a near-uniform mid-gray. Console shows symmetric range around 0 before normalisation.

Example 3: Butterworth LP — tune order

```

import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img); Fs = np.fft.fftshift(F)
h, w = img.shape
cy, cx = h // 2, w // 2

```

```

y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)
D0 = 40

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
for i, n in enumerate([1, 2, 8]):
    mask = 1.0 / (1.0 + (D / D0)**(2 * n))
    out = np.clip(np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask))), 0,
255).astype(np.uint8)
    ax[i].imshow(out, cmap="gray"); ax[i].set_title(f"Butterworth n={n}");
ax[i].axis("off")
plt.tight_layout(); plt.show()

```

Expected Output: n=1 is a very gentle roll-off (close to Gaussian); n=8 is sharper (close to ideal, mild ringing creeping in).

Example 4: D0 sensitivity (Gaussian LP)

```

import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img); Fs = np.fft.fftshift(F)
h, w = img.shape
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)

for D0 in [10, 30, 80, 200]:
    mask = np.exp(-(D**2) / (2 * D0**2))
    out = np.clip(np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask))), 0,
255).astype(np.uint8)
    cv2.imwrite(f"lp_D{D0}.png", out)

```

Expected Output: D0=10 is heavily smoothed (almost flat). D0=200 looks nearly identical to the original (most frequencies kept).

4. Parameter Sensitivity

Param	Effect
D0 (cutoff radius)	small = aggressive filter; large = mild
Mask shape (ideal/Gaussian/Butterworth)	ideal rings; Gaussian smooth; Butterworth tunable
Butterworth order n	1 (gentle) → 8 (near-ideal)

5. Common Mistakes

Mistake	What Happens	Fix
Forget ifftshift before ifft2	Image appears shifted by half	Always undo fftshift
Apply ideal mask without expecting ringing	Halos around edges	Use Gaussian / Butterworth
Mismatched shapes	Broadcast error	Build mask with img.shape

Discard imaginary part incorrectly	Lose phase info	<code>np.real(fft2(...))</code> only after multiplication
------------------------------------	-----------------	---

6. Practice Tasks

- Apply Gaussian LP with $D_0=40$ to `camera()`. Compare to `cv2.GaussianBlur`.
- Apply Gaussian HP with $D_0=30$. Compare to a Laplacian.
- Try Butterworth $n \in \{1, 2, 4, 8\}$ and observe ringing onset.

7. Key Takeaways

- Filter in frequency = multiply spectrum by a mask.
- Ideal mask $\rightarrow$ ringing; Gaussian $\rightarrow$ smooth; Butterworth $\rightarrow$ tunable steepness.
- D_0 is the cutoff radius in spectrum pixels.
- Always `fftshift` before `fft2`.

8. What's Next

Notch filters — surgically remove single periodic noise patterns.

Lesson 4: Notch Filters — Periodic Noise

Module: 14 — Frequency Domain (DFT)

Duration: 30 minutes

Difficulty: Advanced

Learning Objectives

- Identify periodic noise from its spectrum.
- Build notch-reject filters that zero out specific frequency dots.
- Remove screen/scanner moiré with a notch filter.

Prerequisites

- Module 14 Lesson 3

1. Why This Matters

Some noise has structure — scanned magazines have screen patterns, old TVs have horizontal scan lines, fluorescent lights cause banding. These show up as bright dots off-centre in the spectrum, exactly the kind of thing a notch filter is designed to surgically kill.

2. Concept

2.1 Periodic noise → spectrum dots

A sine pattern of frequency (u_0, v_0) in the image appears as two bright dots at (u_0, v_0) and $(-u_0, -v_0)$ (mirrored about DC) in the spectrum. Multiple periodic components → multiple dot pairs.

2.2 Notch reject filter

Zero out small circles around each noise dot, leave the rest of the spectrum alone:

```
def notch_reject(shape, centers, radius=10):
    h, w = shape
    cy, cx = h // 2, w // 2
    y, x = np.ogrid[:h, :w]
    mask = np.ones(shape, dtype=np.float32)
    for u, v in centers:
        # mask out a disk around (cy+v, cx+u) AND its mirror
        d1 = np.sqrt((y - (cy + v))**2 + (x - (cx + u))**2)
        d2 = np.sqrt((y - (cy - v))**2 + (x - (cx - u))**2)
        mask[d1 < radius] = 0
        mask[d2 < radius] = 0
    return mask
```

centers is a list of (u, v) offsets from DC where you spotted noise dots.

2.3 Smooth notches (Gaussian)

A hard disk causes mild ringing too. Smooth version:


```
mask *= 1 - np.exp(-(d1**2) / (2 * radius**2))
```

Replace the assignment with multiplication and a Gaussian — the notch becomes a smooth dip.

2.4 Finding the centres

- Display the log-magnitude spectrum.
- Visually locate bright dots that aren't on the centre axis or expected edges.
- Read their (u, v) offsets from the centre.

For programmatic detection: threshold the spectrum, find connected components, exclude the central blob.

2.5 Common periodic patterns

Source	Pattern
Scanned print	Diagonal pair of dots (the screening pattern)
Halftone newsprint	A rosette / multiple dot pairs
Old TV scan lines	Vertical strip of dots (one along the v-axis)
Power-line interference	Single horizontal dot
CCD readout patterns	Faint vertical band

2.6 When to use notch vs LP

- Periodic noise: notch filter only — LP would also kill the image's high-frequency detail.
- Random noise (grain): use spatial filters (M7) or non-local means (M15).
- Mixed: notch first, then a mild denoiser.

3. Code Examples

Example 1: Add synthetic periodic noise, then remove it

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
h, w = img.shape

# Add a sinusoidal pattern (diagonal stripes at frequency ~10)
yy, xx = np.mgrid[:h, :w]
noise = 30 * np.sin(2 * np.pi * (xx + yy) / 16)
noisy = np.clip(img + noise, 0, 255)

cv2.imwrite("striped.png", noisy.astype(np.uint8))

# Spectrum
F = np.fft.fft2(noisy)
Fs = np.fft.fftshift(F)
mag = np.log1p(np.abs(Fs))
mag_vis = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("striped_spectrum.png", mag_vis)
print("inspect striped_spectrum.png – note the symmetric bright dots off DC")
```

Expected Output (visual): striped.png is the cameraman with diagonal stripes overlaid.
striped_spectrum.png shows the usual centre cross PLUS a pair of bright dots symmetric about the centre — the noise frequencies.

Example 2: Apply a notch filter to remove the stripes

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
h, w = img.shape
yy, xx = np.mgrid[:h, :w]
noisy = np.clip(img + 30 * np.sin(2 * np.pi * (xx + yy) / 16), 0, 255)

F = np.fft.fft2(noisy)
Fs = np.fft.fftshift(F)
cy, cx = h // 2, w // 2

# 16-pixel period diagonal => spectrum frequency at (h/16, w/16) and mirror
# (approximate; exact location depends on resolution)
y, x = np.ogrid[:h, :w]
def notch_disk(u, v, r=8):
    d1 = np.sqrt((y - (cy + v))**2 + (x - (cx + u))**2)
    d2 = np.sqrt((y - (cy - v))**2 + (x - (cx - u))**2)
    return (d1 < r) | (d2 < r)

mask = np.ones((h, w), dtype=np.float32)
mask[notch_disk(32, 32, r=8)] = 0 # tune offsets visually if needed

cleaned = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))
cleaned = np.clip(cleaned, 0, 255).astype(np.uint8)
cv2.imwrite("destriped.png", cleaned)
print("if stripes remain, adjust the (u, v) offsets")
```

Expected Output (visual): destriped.png is the cameraman without (or with greatly reduced) diagonal stripes.

Example 3: Gaussian smoothing on notches — kills ringing

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
h, w = img.shape
yy, xx = np.mgrid[:h, :w]
noisy = np.clip(img + 30 * np.sin(2 * np.pi * (xx + yy) / 16), 0, 255)

F = np.fft.fft2(noisy); Fs = np.fft.fftshift(F)
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]

mask = np.ones((h, w), dtype=np.float32)
for u, v in [(32, 32), (-32, -32)]:
    D = np.sqrt((y - (cy + v))**2 + (x - (cx + u))**2)
```

```

mask *= 1 - np.exp(-D**2 / (2 * 6**2))

cleaned = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))
cleaned = np.clip(cleaned, 0, 255).astype(np.uint8)
cv2.imwrite("destriped_gauss.png", cleaned)

```

Expected Output (visual): Same destriping as Example 2 but no ringing artefacts around edges — softer / cleaner.

Example 4: Interactive workflow

```

# 1. Display log-magnitude spectrum.
# 2. Identify dot pairs that don't belong (not on the cross / not DC).
# 3. Read their (u, v) coordinates (offsets from centre).
# 4. Build a notch mask with `notch_disk(u, v)` for each pair.
# 5. Apply and inspect.
# 6. Iterate.
print("(no code – this lesson's recipe in narrative form)")

```

4. Parameter Sensitivity

Param	Effect
Notch radius	small = surgical (precise); large = also kills nearby useful frequencies
Number of notches	more = handles complex patterns but risk killing real content
Soft (Gaussian) vs hard (disk)	soft = no ringing; hard = simpler to reason about

5. Common Mistakes

Mistake	What Happens	Fix
Forget the mirror pair	Noise only half-removed	Always notch (u, v) AND (-u, -v)
Notch too large	Lose real image content too	Shrink radius
Locate dot wrong	No improvement	Re-inspect spectrum; tune (u, v)
Hard notch on photo	Ringing along edges	Use Gaussian-shaped notches

6. Practice Tasks

- Add diagonal sinusoid noise to camera(); inspect spectrum; locate dot positions.
- Build a notch filter at those positions; remove the stripes.
- Switch to Gaussian notches with a wider σ ; compare to hard disks.

7. Key Takeaways

- Periodic noise $\rightarrow$ bright dots off-centre in the spectrum.
- Notch reject: zero (or soften) small regions around the dots.
- Always mirror about DC for real images.
- Soft (Gaussian) notches prevent ringing.

8. What's Next

The big idea: spatial filters and frequency filters are mathematically the same operation in different domains.

Lesson 5: Spatial ↔ Frequency Equivalence

Module: 14 — Frequency Domain (DFT)

Duration: 25 minutes

Difficulty: Advanced

Learning Objectives

- State the convolution theorem (spatial convolution = frequency multiplication).
- Verify it numerically with NumPy.
- Decide when to filter in spatial vs frequency domain.

Prerequisites

- Module 14 Lessons 1-4, Module 7 (convolution).

1. Why This Matters

Knowing that Gaussian blur in space is the same as multiplying by a Gaussian in frequency is the unifying insight of this module. It explains why spatial filters look the way they do and tells you exactly when to switch domains.

2. Concept

2.1 The convolution theorem

spatial:	$\text{out} = \text{kernel} \otimes \text{image}$	(convolution)
frequency:	$\text{OUT} = \text{KERNEL} \cdot \text{IMAGE}$	(element-wise product)

If you take a kernel's DFT and the image's DFT, multiply, then IFFT — you get the same result as convolving the image with the kernel in the spatial domain.

In Python:

```
F_img = np.fft.fft2(img)
F_kernel = np.fft.fft2(kernel, s=img.shape) # pad kernel to image size
out = np.real(np.fft.ifft2(F_img * F_kernel))
```

(Note: `s=img.shape` zero-pads the kernel so the DFTs have matching shapes.)

2.2 What this tells you intuitively

Spatial filter	Frequency interpretation
Gaussian blur	Low-pass: multiply by a centred Gaussian in spectrum
Box blur	Low-pass: multiply by a sinc (which causes ringing!)
Sharpen kernel	High-pass boost: amplify away from DC
Laplacian	High-pass: zero at DC, growing with frequency

This is why box blur looks slightly boxy — its frequency response is a sinc with side-lobes that let some high frequencies leak through asymmetrically.

2.3 When to pick which domain

Use frequency domain when	Use spatial domain when
Custom shape (notches, band-pass, rings)	Standard small kernels (3×3, 5×5)
Very large kernel ($\geq 30 \times 30$)	Small kernel — spatial faster
You want to design by drawing in spectrum	You want speed and simplicity
Educational / debugging	Production code (usually)

Rule of thumb: for kernels under $\sim 21 \times 21$, spatial is faster (less overhead). For huge kernels or non-standard shapes, frequency wins.

2.4 Periodic boundary

fft2 assumes the image is periodic — i.e. wraps around. If your image's edges don't match, you'll see slight ringing near the borders. Two fixes:

- Pad before FFT (zero or reflect), crop after.
- Use cv2.dft with cv2.DFT_REAL_OUTPUT and OpenCV's optimal size: cv2.getOptimalDFTSize(N).

2.5 Closing the loop

You can now read any spatial filter as a frequency operation, and vice versa. Practical takeaway: when a spatial filter behaves unexpectedly (ringing, aliasing, halo), look at its DFT — the answer is usually visible there.

3. Code Examples

Example 1: Numerically verify the convolution theorem

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
kernel = np.array([[0, -1, 0],
                  [-1, 5, -1],
                  [0, -1, 0]], dtype=np.float32)

# Spatial
spatial = cv2.filter2D(img, -1, kernel, borderType=cv2.BORDER_CONSTANT)

# Frequency
F_img = np.fft.fft2(img)
F_k = np.fft.fft2(kernel, s=img.shape)
freq = np.real(np.fft.ifft2(F_img * F_k))

# Align: filter2D centres the kernel; the freq method places it at (0,0)
# We can compare after recentering or just check inner regions.
spatial = np.clip(spatial, 0, 255).astype(np.uint8)
freq = np.clip(freq, 0, 255).astype(np.uint8)
```

```
print("spatial range:", spatial.min(), spatial.max())
print("freq    range:", freq.min(), freq.max())
```

Expected Output: Both produce a sharpened image. Exact pixel-by-pixel match requires careful centring of the kernel; the qualitative result is identical.

Example 2: Gaussian blur — equivalent in both domains

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
sigma = 3.0

# Spatial
spatial = cv2.GaussianBlur(img, (0, 0), sigma)

# Frequency: multiply by Gaussian in spectrum
F = np.fft.fft2(img); Fs = np.fft.fftshift(F)
h, w = img.shape; cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)
#  $\sigma$  in image space  $\rightarrow \sigma' \approx N/(2\pi \sigma)$  in frequency space (for a 1D case)
freq_sigma = max(1.0, min(h, w) / (2 * np.pi * sigma))
mask = np.exp(-D**2 / (2 * freq_sigma**2))
freq = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))

print("max abs diff (loose):",
      np.abs(spatial.astype(np.float32) - freq.astype(np.float32)).max())
```

Expected Output: Diff is non-zero but small. Both are visibly the same Gaussian blur. (Exact agreement requires careful boundary handling; this demonstrates the principle.)

Example 3: Show that box blur has sinc response

```
import numpy as np
import matplotlib.pyplot as plt

k = np.ones(21) / 21 # 21-wide box kernel (1D)
K = np.abs(np.fft.fft(k, n=512))
K = np.fft.fftshift(K)

plt.plot(K)
plt.title("DFT of box kernel — sinc-like ripples")
plt.xlabel("frequency bin"); plt.ylabel("|K|"); plt.show()
print("note the oscillating side-lobes — they cause aliasing")
```

Expected Output: A sinc-shaped curve with several oscillating side-lobes. That's why box blur looks "boxy" — the side-lobes admit some high frequencies and reject others non-monotonically.

Example 4: Timing — when does frequency win?

```
import cv2, numpy as np, time
from skimage.data import astronaut
```

```

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2GRAY).astype(np.float32)
img = cv2.resize(img, (1024, 1024))

for ks in [5, 21, 101]:
    k = np.ones((ks, ks), np.float32) / (ks * ks)

    t = time.perf_counter()
    cv2.filter2D(img, -1, k)
    t_sp = (time.perf_counter() - t) * 1000

    t = time.perf_counter()
    F_img = np.fft.fft2(img)
    F_k = np.fft.fft2(k, s=img.shape)
    _ = np.real(np.fft.ifft2(F_img * F_k))
    t_fq = (time.perf_counter() - t) * 1000

    print(f"ksize={ks:3d}: spatial={t_sp:6.1f} ms frequency={t_fq:6.1f} ms")

```

Expected Output (timings vary):

```

ksize=  5: spatial=  2.0 ms frequency=  90.0 ms
ksize= 21: spatial= 25.0 ms frequency=  88.0 ms
ksize=101: spatial= 400.0 ms frequency=  90.0 ms

```

Spatial wins for small kernels; frequency wins as kernel grows.

4. Parameter Sensitivity

Param	Effect
Kernel size	small → spatial faster; large → frequency faster
Image size	bigger images make the frequency overhead more amortised

5. Common Mistakes

Mistake	What Happens	Fix
Forget to pad kernel to image size in fft2	Shape mismatch	<code>np.fft.fft2(k, s=img.shape)</code>
Compare DFT result without recentering	Pixel-wise mismatch but qualitative match	Centre the kernel or compare cropped interiors
Assume frequency is always faster	Small kernels are slower in frequency	Profile first

6. Practice Tasks

- Apply a 3×3 sharpening kernel via both `cv2.filter2D` and frequency multiplication; compare qualitatively.
- Plot the DFT magnitude of a 21-wide box kernel; note the side-lobes.
- Time spatial vs frequency for kernel sizes 5/21/101 on a 1024² image.

7. Key Takeaways

- Convolution theorem: spatial convolution = frequency multiplication.
- Pad kernel to image size for the FFT product.
- Frequency wins on large or custom-shape kernels; spatial wins on small standard kernels.
- The "boxy" look of box blur is its sinc-shaped frequency response.

8. What's Next

End of Module 14. Next: Module 15 — Image Restoration & Noise.

Module 14 — Exercises

15 exercises.

14.1 1D Fourier

- (E) Build a 256-sample signal with sines at frequencies 8 and 20; FFT and identify peaks.
- (M) Add Gaussian noise to a 1D sine; recover it by zeroing high-frequency bins.
- (H) Inspect the DFT of one row of `camera()`; print the DC magnitude.

14.2 2D DFT

- (E) Compute and log-display the spectrum of `camera()`.
- (M) Compare spectra of original vs Gaussian-blurred ($\sigma=4$) vs noisy versions.
- (H) Use `cv2.dft` instead of `np.fft.fft2`; verify magnitudes match.

14.3 LP / HP filters

- (E) Apply a Gaussian LP with $D_0=40$ to `camera()`; compare to `cv2.GaussianBlur`.
- (M) Apply a Gaussian HP with $D_0=30$; compare visually to a Laplacian edge map.
- (H) Try Butterworth orders $n \in \{1, 2, 4, 8\}$; describe ringing onset.

14.4 Notch

- (E) Add a diagonal sinusoidal noise pattern to `camera()`; inspect spectrum.
- (M) Locate the noise dots; build a hard-disk notch filter; remove the stripes.
- (H) Switch to Gaussian-shaped notches; compare ringing.

14.5 Equivalence

- (E) Apply a 3×3 sharpen kernel via both `cv2.filter2D` and FFT-multiply.
- (M) Plot the DFT magnitude of a 21-wide 1D box kernel; identify side-lobes.
- (H) Time spatial vs frequency for kernel sizes 5/21/101 on a 1024^2 image.

Module 14 — Quiz

10 MCQs.

Q1

The DC (zero frequency) component of a DFT represents: A. the brightest pixel B. the image's average value $\times N$ C. random noise D. the edges

Answer: B (L1, L2)

Q2

`np.fft.fftshift` is used to: A. compress B. centre the DC in the spectrum for visualisation C. invert D. blur

Answer: B (L1, L2)

Q3

For real-valued images, the spectrum is: A. random B. Hermitian-symmetric about the centre C. always real D. always positive

Answer: B (L2)

Q4

The visual cross pattern in a spectrum corresponds to: A. periodic noise B. strong horizontal and vertical edges in the image C. DC term D. blur

Answer: B (L2)

Q5

An ideal LP filter causes: A. blur with no artefacts B. ringing (sinc artefacts) C. sharpening D. noise

Answer: B (L3)

Q6

For a smooth, ringing-free LP, the natural mask shape is: A. ideal disk B. Gaussian C. random D. uniform

Answer: B (L3)

Q7

Periodic noise (e.g. screen pattern) appears in the spectrum as: A. uniform haze B. bright dots off-centre, symmetric pairs C. central spike D. radial lines

Answer: B (L4)

Q8

A notch filter: A. zeros (or attenuates) small regions around noise peaks B. blurs everywhere C. only acts on DC D. is the inverse of LP

Answer: A (L4)

Q9

The convolution theorem says: A. spatial convolution = frequency multiplication B. they're unrelated C. only Gaussians qualify D. only with DCT

Answer: A (L5)

Q10

For a 5×5 kernel on a 512^2 image, the faster domain is usually: A. spatial B. frequency C. either — same D. parallel only

Answer: A (L5)

Module 14 — Assignment: Frequency-Domain Denoiser

Estimated time: 2 hours

Combines: Module 14 + Module 7

Brief

Build a CLI that denoises an image using frequency-domain filtering and compares the result against an equivalent spatial-domain filter.

Requirements

- `freq_denoise.py INPUT --mode lp|hp|notch [--D0 40] [--notch-uv u,v[;u,v...]] [--output OUT]`
- lp mode: Gaussian low-pass with cutoff D0. Output = LP result.
- hp mode: Gaussian high-pass with cutoff D0. Output = HP-only image (normalised for display).
- notch mode: notch filter at each (u, v) plus its mirror (radius 8, Gaussian shape).
- Always also produce a spatial baseline:
- For lp → `cv2.GaussianBlur` with σ matched to D0.
- For hp → spatial high-pass = original – `GaussianBlur`.
- For notch → spatial baseline = original (no easy equivalent — note in output).
- Save a 3-up side-by-side: [original | frequency-result | spatial-baseline].
- Print PSNR (frequency vs spatial) and any other stats you choose.

Constraints

- Modules 1-14 only.
- Use `np.fft.fft2` / `ifft2` and `np.fft.fftshift` / `ifftshift`.
- Use `argparse`.
- For `--notch-uv`, parse "32,32;40,10" into a list of (u, v) pairs.

Expected Output

```
$ python freq_denoise.py noisy.png --mode lp --D0 30
mode: lp D0: 30
PSNR(freq vs spatial): 42.1 dB
saved -> noisy_freq.png
```

Grading Rubric

Criterion	Weight
FFT + shift pipeline	25%
Three modes correct	30%
Spatial baseline	15%
3-up output	15%
Argparse + style	15%

Submission

Save as assignment_module14.py.

Module 14 — Mini Project: Interactive Frequency Filter Tuner

Overview

An interactive cv2-window tool: load an image, see its log-magnitude spectrum side-by-side with the filtered result, and tune cutoff D_0 , filter type (LP / HP / Band-pass), and notch positions live.

Concepts Used

- 2D DFT (M14 L2)
- LP / HP / Butterworth masks (M14 L3)
- Notch filters (M14 L4)
- Trackbars (M2 L3)

Features

- Trackbars: D_0 , mode (LP/HP/BP/notch), Butterworth order n , notch radius.
- 3-panel display: original | filtered | spectrum (log).
- `s` saves the filtered image; `q` quits.

Starter Code

freq_tuner.py:

```
import sys, cv2, numpy as np

MODES = ["LP", "HP", "BP", "notch"]

def filter_freq(img, mode, D0, n, notch_radius):
    f = img.astype(np.float32)
    F = np.fft.fft2(f)
    Fs = np.fft.fftshift(F)
    h, w = f.shape
    cy, cx = h // 2, w // 2
    y, x = np.ogrid[:h, :w]
    D = np.sqrt((y - cy)**2 + (x - cx)**2)

    if mode == "LP":
        mask = 1 / (1 + (D / max(1, D0))**(2 * max(1, n)))
    elif mode == "HP":
        mask = 1 - 1 / (1 + (D / max(1, D0))**(2 * max(1, n)))
    elif mode == "BP":
        lp1 = 1 / (1 + (D / max(1, D0))**(2 * max(1, n)))
        lp2 = 1 / (1 + (D / max(1, int(D0 * 1.6)))**(2 * max(1, n)))
        mask = lp2 - lp1 # band-pass
    else: # notch - fixed diagonal pair as a demo
        u, v = int(D0), int(D0)
        mask = np.ones((h, w), dtype=np.float32)
        for du, dv in [(u, v), (-u, -v)]:
            d = np.sqrt((y - (cy + dv))**2 + (x - (cx + du))**2)
            mask *= 1 - np.exp(-d**2 / (2 * max(1, notch_radius)**2))
```

```

out = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))
out = cv2.normalize(out, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
spec = cv2.normalize(np.log1p(np.abs(Fs * mask)), None, 0, 255,
                    cv2.NORM_MINMAX).astype(np.uint8)

return out, spec

def run(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 600: img = cv2.resize(img, (512, 512))

    cv2.namedWindow("tuner")
    cv2.createTrackbar("D0", "tuner", 40, 250, lambda v: None)
    cv2.createTrackbar("mode 0LP 1HP 2BP 3notch", "tuner", 0, 3, lambda v: None)
    cv2.createTrackbar("order n", "tuner", 2, 8, lambda v: None)
    cv2.createTrackbar("notch radius", "tuner", 8, 30, lambda v: None)

    while True:
        D0 = cv2.getTrackbarPos("D0", "tuner")
        m = MODES[cv2.getTrackbarPos("mode 0LP 1HP 2BP 3notch", "tuner")]
        n = cv2.getTrackbarPos("order n", "tuner")
        nr = cv2.getTrackbarPos("notch radius", "tuner")

        out, spec = filter_freq(img, m, D0, n, nr)
        triple = np.hstack([img, out, spec])
        cv2.putText(triple, f"{m} D0={D0} n={n} notch_r={nr}",
                    (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.6, 255, 2)
        cv2.imshow("tuner", triple)
        k = cv2.waitKey(30) & 0xFF
        if k == ord('q'): break
        if k == ord('s'): cv2.imwrite("freq_out.png", out); print("saved
freq_out.png")
        cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/camera.png")

```

Expected Interaction

Move D0 to change cutoff; switch mode to see LP/HP/BP/notch effects; spectrum panel updates with the masked spectrum.

Extension Challenges

- Add a mouse-click handler to place notch centres interactively.
- Add a "spatial-equivalent" baseline panel for comparison.
- Save the mask alongside the result.

Grading Rubric

Criterion	Weight
Four modes work	35%
Spectrum panel updates	20%

Trackbars responsive	15%
Save / quit	10%
Style	20%